Computer-System Organization and Architecture — Lecture Notes — Week 2

Interrupts, Storage Structure, I/O Structure, and DMA

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 17, 2026

Concepts That Click

2.1 Introduction: The Machine the OS Must Manage

Suppose you press `Enter` to run `cat bigfile.txt`. Before a single byte reaches the screen, the keyboard controller, the CPU, the memory, the disk controller, and the system bus all have to cooperate. A natural question follows: which of these components can signal the CPU, and how do they do it? The answer is the subject of this chapter. Last week we asked what the operating system *is*. This week we examine the machine it stands on, and the two mechanisms — **interrupts** and **DMA** — that make it possible for the OS to stand there at all.

The central observation is that the OS does not run "on its own." It is **event-driven**: it reacts to hardware events it did not initiate. Something else — a key press, a packet arrival, a disk completing a transfer — must first give the kernel the CPU. Understanding the machine therefore means understanding who can interrupt whom, where data live, and how bytes actually move.

Five questions organize the chapter:

1. **What is the machine made of?** — CPU, memory, controllers, bus.
2. **How does hardware get the CPU's attention?** — interrupts, traps, and the interrupt vector.
3. **Where do data live?** — main memory, disks, SSDs, and the memory hierarchy.
4. **How do bytes move?** — polling, interrupt-driven I/O, and DMA.
5. **How does the machine scale?** — multiprocessors, multicore, and clusters.

Every answer is an OS-visible fact: each mechanism in this chapter reappears later as a scheduling, memory, or I/O policy. Our running thread is the command `cat bigfile.txt`, followed from the Enter key to the disk platter and back: an interrupt starts it, a storage hierarchy feeds it, and DMA moves the bytes.

2.2 Anatomy of a Computer System

A computer is not a CPU that does everything. It is a set of independent devices that run concurrently and communicate over a shared bus. Figure 2.1 shows the arrangement: a CPU, main

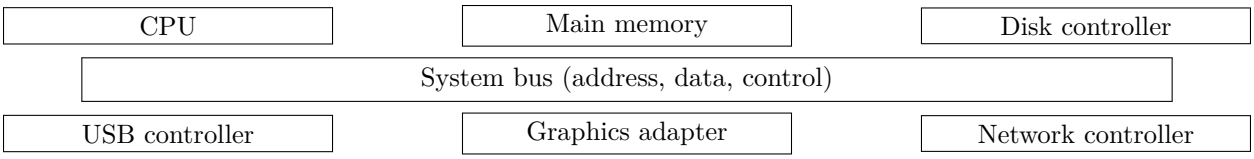


Figure 2.1: Anatomy of a computer system: independent devices attached to a shared system bus.

memory, and a disk controller above the bus, and a USB controller, a graphics adapter, and a network controller below it.

Definition (Device controller). *Hardware that speaks the bus protocol and operates a physical device. A controller exposes registers the CPU can read and write, and it runs concurrently with the CPU.*

Each device has a controller, and the CPU and controllers run **concurrently**. The bus is the shared resource they contend for. This single structural fact explains most of what follows: because the devices are independent and the bus is shared, the OS must have a way to start an operation on one device and be notified later, without watching the device in the meantime.

2.2.1 Bootstrap: How the Machine Starts

The machine does not begin with an operating system in memory. The sequence is:

1. **Firmware** (BIOS or UEFI) runs from ROM or flash when power is applied. It tests the hardware with a power-on self-test (POST) and finds a boot device.
2. The **bootstrap program** loads the OS kernel from disk into memory and jumps to its entry point.
3. The kernel initializes its data structures, starts the first process (`init` or `systemd`), and from then on is **event-driven**.

The important consequence is that the OS cannot assume it is running. Until the kernel is loaded, no OS exists. After it is loaded, the kernel only runs when a *trap*, an *interrupt*, or an explicit call gives it the CPU. The rest of this chapter is about that handoff (Silberschatz et al., Ch. 1, pp. 6–8; Stallings, Ch. 1 [2]).

2.2.2 The CPU's View: Fetch, Decode, Execute — and Wait

The CPU repeats one cycle: **fetch** an instruction at the program counter, **decode** it, **execute** it, and update the program counter. Its registers hold the working set: general-purpose registers, the **program counter (PC)**, the stack pointer, and the status word.

The problem is speed. I/O is orders of magnitude slower than the CPU. A 3 GHz core executes roughly three instructions per nanosecond, whereas a disk access takes milliseconds. If the CPU waited for every device, it would idle almost all the time. The OS therefore needs a way to *start* an I/O operation and be told later that it finished — without watching it (Stallings, Ch. 1 §1.2–1.3, pp. 7–15, [2]).

2.3 Controllers and Drivers

Every device has two faces, one in hardware and one in software.

Concepts That Click

Definition (Device controller (hardware)). *A chip or board with registers the CPU can read and write:* ***status*** *(busy, ready, error),* ***control*** *(start, reset, mode), and* ***data-in/data-out*** *buffers. It operates the physical device.*

Definition (Device driver (software)). *Kernel code that knows the controller's register layout and protocol. The OS calls the driver; the driver programs the controller; the controller moves the data.*

There is one driver per controller family, and the rest of the kernel sees a uniform interface (read, write, ioctl). The kernel also keeps a **device-status table** listing each device, its type, address, and state; Section 2.6 uses that table when an interrupt arrives (Silberschatz et al., Ch. 1, pp. 6–18).

2.3.1 Worked Thread: The Enter Key, Step by Step

Example 2.1. *You press* `Enter` *to run* `cat bigfile.txt`*. Trace what happens before the shell starts.*

1. *The keyboard controller scans the key matrix and places a scan code in its data register.*
2. *The controller asserts an* ***interrupt request (IRQ)*** *line on the bus.*
3. *The CPU finishes the current instruction, saves its state, and jumps to the keyboard* ***interrupt service routine (ISR)****.*
4. *The ISR reads the scan code, maps it to a character, and queues it for the terminal process.*
5. *The scheduler eventually runs the shell, which reads the character and starts* `cat bigfile.txt` *— which now needs the disk.*

The pattern to remember is: device → controller → IRQ → CPU saves state → ISR → kernel decides what happens next. Every device in the machine uses a variant of this path.

2.4 Interrupts and Traps

There are two kinds of transfer of control into the kernel, and the difference is timing.

Definition (Interrupt (asynchronous)). *A transfer raised by* hardware *at an unpredictable time: a key press, a packet arrival, a disk completing a transfer. It is unrelated to the instruction the CPU is currently executing.*

Definition (Trap or exception (synchronous)). *A transfer raised by the* CPU *because of the instruction just executed: a system call, a divide by zero, or an illegal opcode. It is caused by the program itself.*

Both transfer control to a kernel handler and both save the CPU state. The distinction matters because an interrupt can arrive *between* any two instructions, whereas a trap is tied to a specific instruction.

2.4.1 The Interrupt Path

Figure 2.2 shows the route from a device to a handler. The device asserts an IRQ; an interrupt controller prioritizes and forwards it; the CPU dispatches through a table of handler addresses.

The path unfolds in four steps:

Concepts That Click

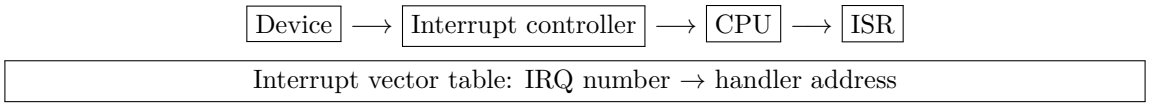


Figure 2.2: The interrupt path: device, interrupt controller, CPU, and interrupt service routine.

1. The device asserts an IRQ; the interrupt controller prioritizes it and forwards it to the CPU.
2. The CPU finishes the current instruction (not the whole program), saves the PC and status word, and switches to kernel mode.
3. The CPU indexes the **interrupt vector table** by IRQ number and jumps to the handler.
4. The ISR services the device; on return, the saved state is restored.

2.4.2 Vector Table, Masking, and Priority

The **interrupt vector table** is an array of handler addresses indexed by interrupt number. It lets the CPU dispatch without searching: the IRQ number itself is the index. **Maskable** interrupts can be temporarily disabled by the kernel while it updates shared structures; **non-maskable** interrupts (a power failure, a fatal hardware error) cannot. **Priority levels** decide which interrupt wins when several arrive at once, and a low-priority handler can itself be interrupted by a high-priority one.

There is a design tension here. Handlers must be short, or they block other interrupts and add latency. Real kernels therefore split the work: a minimal **top half** acknowledges the device, and a deferred **bottom half** does the rest (Linux softirqs, tasklets, and work queues) (Bovet & Cesati, Ch. 4, pp. 131–188, [4]).

This also answers a natural objection: what if a handler runs with interrupts temporarily disabled and is itself long? Other devices cannot deliver their interrupts, so their data can be overwritten or lost — a **missed interrupt** — and the system's **interrupt latency** grows until a real-time device may miss its deadline. Hence the top-half/bottom-half split: acknowledge quickly, then defer the heavy work to a context where interrupts are enabled.

2.4.3 Worked Example: What Does Interrupt Handling Cost?

Example 2.2. *Saving the context and entering the handler costs* 2 μs*; the handler itself runs for* 8 μs*. Each interrupt therefore costs* 10 μs *of CPU time. Compute the CPU overhead at three interrupt rates.*

*At **1,000 interrupts/s**:*

$$1{,}000 \times 10\ \mu s = 10\ ms/s = 1\%\ \textit{CPU overhead.}$$

*At **10,000 interrupts/s**:*

$$10{,}000 \times 10\ \mu s = 100\ ms/s = 10\%\ \textit{CPU overhead.}$$

*At **100,000 interrupts/s**, the machine spends* 100% *of its time handling interrupts and does no useful work.*

The lesson is that interrupts are cheap per event but not free. When a device produces a stream *of events, per-event interrupts stop scaling — which is exactly the problem DMA solves for block transfers.*

Concepts That Click

2.5 Storage Structure and the Memory Hierarchy

Interrupts tell the CPU that data has arrived. The next question is where the data live, and why a program can believe it has more memory than the machine owns.

Definition (Main memory (RAM)). *A large array of bytes, each with an address, implemented in DRAM. It is fast (tens of nanoseconds) and directly addressable by the CPU, but it is volatile: its contents vanish when power is lost.*

Definition (Secondary storage). ***Hard disks*** *(magnetic platters) and* ***solid-state drives*** *(NAND flash). It is non-volatile and far larger than RAM, but orders of magnitude slower — microseconds for an SSD and milliseconds for a spinning disk.*

Programs live in memory only while they run; files persist on secondary storage. The OS moves data between the two in units of **blocks** (on disk) or **cache lines** (in memory).

2.5.1 Hard-Disk Geometry: Where Access Time Comes From

A hard disk is organized around rotating **platters**. Each surface has concentric **tracks** split into **sectors**, and the same track across platters is a **cylinder**. Access time has three components:

- **Seek**: move the arm to the track (mechanical, milliseconds).
- **Rotational latency**: wait for the sector to pass under the head (milliseconds).
- **Transfer**: stream the requested sectors (microseconds per block).

In symbols, total access time = seek + rotational latency + transfer.

Example 2.3. *A disk has seek time* 8 *ms, rotation speed* 7,200 *RPM, transfer rate* 150 *MB/s, and a block size of* 4 *KB. Compute the time for one block read.*

One rotation takes

$$\frac{60}{7{,}200}\ s = 8.33\ ms,$$

so the average rotational latency is half of that, $4.17\ ms$*. The transfer time is*

$$\frac{4\ KB}{150\ MB/s} \approx 27\ \mu s.$$

The total is therefore

$$8 + 4.17 + 0.027 \approx 12.2\ ms,$$

of which 99.8% *is mechanical waiting.*

The consequence for the OS is that disk performance is dominated by ***where the head is****, not by how fast bytes stream. This is why scheduling the* order *of requests (disk scheduling, later in the course) matters so much.*

2.5.2 The Memory Hierarchy

Storage is not one technology but a hierarchy (Figure 2.3): registers, cache, main memory, solid-state drive, magnetic disk, and tape or optical archive.

Moving down the hierarchy, capacity grows, cost per byte falls, and access time grows by orders of magnitude. Moving up, data is **cached**: a copy is kept in the faster level on the bet that it will be used again.

Concepts That Click

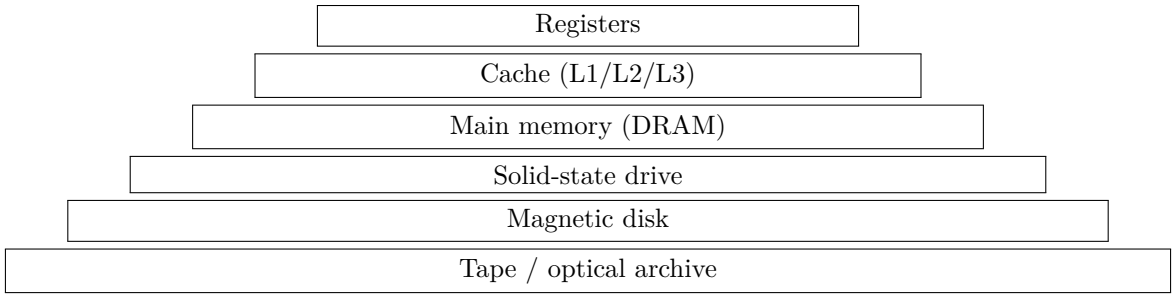


Figure 2.3: The memory hierarchy: capacity grows and cost per byte falls as access time grows.

2.5.3 Locality, Caching, and Why the Hierarchy Works

The hierarchy works because programs do not access memory uniformly. **Temporal locality** means an item used once is likely to be used again soon; **spatial locality** means an item near one just used is likely to be used soon.

Definition (Cache). *A small, fast store holding copies of recently used data. A **hit** is served from the cache; a **miss** goes to the next level and installs a copy.*

The OS manages caches at every level: CPU caches (in hardware), the **buffer cache** for disk blocks, and memory itself as a cache for disk (Stallings, Ch. 1 §1.5–1.6, pp. 26–32, [2]; Silberschatz et al., Ch. 1, pp. 9–14).

A useful check on the main-memory/secondary-storage distinction: main memory is volatile, yet a file you saved last week still exists after a power cut. The file lives on **secondary storage**, which is non-volatile. RAM held the program's buffer and the kernel's **cache** while the write was in progress, and the OS copied the dirty blocks to storage. This is why "save" is an I/O operation, not a memory operation — and why a crash before the write-back can lose data.

2.6 I/O Structure: Polling, Interrupts, and DMA

Data must come from a device into memory. The remaining question is who moves it: the CPU, one word at a time, or a controller acting on its own. There are three techniques, and they form a progression.

2.6.1 Polling: Simple but Wasteful

Definition (Programmed I/O (polling)). *The CPU repeatedly reads the controller's **status register** until the device is ready, then moves the data word by word.*

Polling is simple: it needs no interrupt machinery and no concurrency. It is also wasteful: every poll is a bus transaction and a wasted instruction slot, and a slow device wastes almost all of them. It is still the right choice for very fast devices (a GPU register) or when the wait is predictably short — polling a device that is always ready beats paying interrupt overhead (Silberschatz et al., Ch. 1 §1.2.3, pp. 12–18; Stallings, Ch. 1 §1.7, p. 33, [2]).

This is also why the OS cannot simply poll every device in a loop. There are two distinct costs. The first is **CPU cost**: polling consumes cycles that could run user code, and most polls find nothing ready. The second is **latency cost**: a slow or sporadic device (a keyboard, a network packet) may wait an entire polling period before being noticed. Interrupts invert the arrangement: the device tells the CPU when it needs attention, instead of the CPU asking.

Concepts That Click

2.6.2 Interrupt-Driven I/O: The CPU Is Notified

Interrupt-driven I/O works in four steps:

1. The driver starts the device by writing its control register.
2. The CPU returns to other work; the scheduler runs another process.
3. When the device finishes, it raises an interrupt.
4. The ISR reads the data (or a status), acknowledges the device, and wakes the waiting process.

The cost model is the key: CPU cost scales with the *number of events*, not the number of bytes. Small blocks therefore mean many interrupts; large blocks mean few. This is the lever that DMA pulls.

2.6.3 Direct Memory Access (DMA)

Figure 2.4 shows the DMA arrangement. The CPU programs a DMA controller with the memory address, the byte count, and the device; the controller then transfers the whole block directly to or from memory, stealing bus cycles but no CPU instructions; and it raises *one* interrupt when the block is complete.

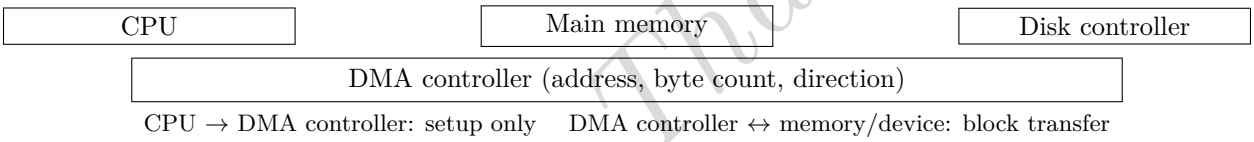


Figure 2.4: Direct memory access: the CPU sets up the transfer, and the controller moves the block.

DMA does not mean that a transfer uses no CPU time at all. There are hidden costs. **Setup**: the CPU programs address, count, and direction for every transfer. **Bus contention**: the DMA controller steals memory cycles, slowing the CPU's own memory accesses (cycle stealing). **Cache coherence**: the CPU may hold stale copies of memory the DMA wrote, so the OS and hardware must invalidate or flush. DMA is therefore a *trade* of per-byte CPU work for per-block setup and bus pressure — not a free lunch (Silberschatz et al., Ch. 1 §1.2.3; Stallings, Ch. 1 §1.7, p. 33, [2]).

2.6.4 Worked Example: Moving 1 MB Three Ways

Example 2.4. *A disk transfers at* 200 *MB/s. An interrupt costs* 10 μs*, and DMA setup plus the completion interrupt costs about* 20 μs*. The transfer time for 1 MB is* 5 *ms. Compare the three techniques.*

Interrupt-driven, 4 KB blocks: 1 *MB*/4 *KB* = 256 *interrupts, so*

$$256 \times 10\ \mu s = 2.56\ ms,$$

about 51% of the transfer time.

Interrupt-driven, 64 KB blocks: 16 *interrupts, so*

$$16 \times 10\ \mu s = 160\ \mu s,$$

about 3.2%.

Concepts That Click

DMA: one setup plus one completion interrupt is about 20 μs, about 0.4%, while the CPU runs other work.

The trade-off, stated precisely: larger blocks amortize interrupt overhead but increase latency and buffer memory. DMA removes per-byte CPU work but adds setup cost — so it wins on large transfers, not on a single byte.

2.6.5 Device Types and the Device-Status Table

Block devices are addressable in fixed-size blocks and are seekable — disks, SSDs, USB drives. They support file systems and DMA. **Character devices** deliver a stream of bytes and are not seekable — keyboards, serial ports, mice. They are naturally interrupt-driven.

The kernel maintains the **device-status table** introduced earlier: for each device, its type, address, and current state (idle, busy, error). When an interrupt arrives, the kernel looks up the device in the table, updates its state, and wakes any process waiting on it (Silberschatz et al., Ch. 1, pp. 12–18).

The three techniques are a progression: the CPU does *more* waiting under polling, *less* work under interrupt-driven I/O, and *no* per-byte work under DMA. `cat bigfile.txt` uses a **block device**, so DMA is the natural choice; a keyboard uses **interrupt-driven** I/O.

2.7 Computer-System Architecture

One CPU, one memory, and one disk is the simple case. Real machines replicate processors and whole systems, and the OS must keep them coherent.

Definition (Single-processor system). *A system with one general-purpose CPU. Most personal computers and phones have several cores per chip, but the OS still sees a set of processors to schedule.*

A **multiprocessor** system has two or more processors that share the bus, clock, and memory. In **symmetric multiprocessing (SMP)**, every processor is equal and runs the same OS, so the scheduler can place any task on any CPU. In an **asymmetric** system, one master processor assigns work to slaves — simpler, but the master is a bottleneck.

Multicore is a different packaging of the same idea: several independent cores on one chip, each with its own registers and usually its own L1/L2 cache, sharing a last-level cache and memory.

A **cluster** goes further: two or more complete systems connected by a network, sharing storage and presenting as one service. Clusters are used for **high availability** (failover) and **high-performance computing** (parallel work).

NUMA (non-uniform memory access) describes large multiprocessors in which memory is attached to nodes. A core accesses local memory faster than a remote node's memory, so the scheduler and allocator should keep a task near its data.

Why does the OS care? Because it must decide *where* to run a task and *where* to place its memory — decisions that do not exist on a single-processor machine. Comparing the three forms: SMP shares the bus, clock, and memory; multicore shares the last-level cache and memory; a cluster shares only the network and storage. The failure unit is one processor, one core, or one whole machine, respectively. All three provide **parallelism**, but only a cluster survives the failure of an entire machine, and all three make **concurrency** real for the OS — the subject of Unit II.

A quantitative check makes the shared-resource point concrete. Suppose a machine has four cores but a single memory bus, and two programs each read a 1 GB array. The **memory bus** is the shared bottleneck: more cores do not add memory bandwidth, so the workload is **memory-bound**, not CPU-bound. The first fixes are to improve **locality** (block the loops so each cache

Concepts That Click

line is reused) and then to add cache or bandwidth — not more cores. This is why the OS and the compiler care about caches and NUMA placement, not just the number of processors.

The recurring theme is that every layer of hardware organization — cache, memory, bus, processor — becomes a policy question for the OS: which copy, which CPU, which node, which order.

2.8 Summary

A computer system is a **CPU, memory, controllers, and a bus** — independent devices that run concurrently. Hardware gets the CPU's attention with an **interrupt** (asynchronous); the CPU gets the kernel's attention with a **trap** (synchronous). The **interrupt vector** dispatches to a handler, and handlers must be short, hence top-half/bottom-half designs.

Storage is a **hierarchy**: registers, cache, RAM, SSD, disk, tape — faster and costlier per byte as you move up, kept coherent by **caching** and locality. I/O moves data by **polling**, **interrupts**, or **DMA**; the choice trades CPU work against latency, overhead, and bus pressure. Finally, **SMP, multicore, and clusters** replicate processors or machines, and the OS must schedule and place work across them.

The running thread can be told in four steps. First, a **keyboard interrupt**: the controller signals, the ISR queues a character, and the shell runs. Second, the **storage hierarchy**: `cat` asks for file blocks, and the buffer cache checks RAM before touching the SSD or disk. Third, **DMA**: the disk controller moves the block into memory, and the CPU is interrupted once when it finishes. Fourth, **architecture**: the scheduler may run `cat` on another core while the DMA transfer proceeds — and must keep the caches coherent.

In one sentence: the OS is an event-driven manager of a hierarchy of asynchronous devices — interrupts tell it *when*, storage tells it *where*, and DMA tells it *how* to move data without doing the moving itself.

Next lecture turns to how the OS *stays in control* — dual-mode operation, the timer, privileged instructions, and the protection boundary that makes every mechanism in this chapter safe to share (Silberschatz et al., Ch. 1 §1.5, pp. 20–23, and §1.2, pp. 6–18, [1]; Stallings, Ch. 1 §1.4–1.7, pp. 15–38, [2]). For optional concrete kernel depth, Bovet and Cesati follow the interrupt and exception path through Linux in Ch. 4 (pp. 131–188) [4].

Self-Check

1. Why can the OS not simply poll every device in a loop? Name two distinct costs of that design.

2. An interrupt handler runs with interrupts temporarily disabled and is long. What can go wrong, and what does this imply about handler design?

3. Main memory is volatile, yet a file saved last week still exists after a power cut. Where is the file, and what role did RAM play in saving it?

4. DMA removes the CPU from the data path. Does that mean a DMA transfer uses no CPU time at all? Name the hidden costs.

5. A machine has four cores but a single memory bus, and two programs each read a 1 GB array. What limits the speedup, and what would you change first?

Concepts That Click

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 8th ed., John Wiley and Sons — Ch. 1 §1.2 (computer-system organization, interrupts, storage, I/O, DMA, pp. 6–18), §1.3 (computer-system architecture, pp. 15–20), §1.7–1.8 (memory and storage management, pp. 24–26). *(Page-cited; index built 2026-10-14.)*

[2] W. Stallings, *Operating Systems: Internals and Design Principles*, 6th ed., Prentice Hall — Ch. 1 §1.4 (interrupts, p. 15), §1.5 (memory hierarchy, p. 26), §1.6 (cache, p. 29), §1.7 (I/O techniques and DMA, p. 33), i.e. pp. 15–38. *(Page-cited; index built 2026-10-14.)*

[3] M. J. Bach, *The Design of the UNIX Operating System*, Prentice Hall of India, 1986.

[4] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed., O'Reilly Media, 2006 — Ch. 4 (interrupts and exceptions, pp. 131–188). *(Page-cited; index built 2026-10-14.)*

Concepts That Click